Optimal Model for Post-Surgery Patient Survival

Load packages

```
library(dplyr)
library(tidyverse)
library(cowplot)
library(ggplot2)
library(scales)  # help reduce output to specified decimal places
library(ggResidpanel)
```

Read the Dataset

```
surgery <- read.csv(file.choose())
```

```
head(surgery)
```

```
##   blood prog enzyme liver age gender a.mod a.heavy survival
## 1   6.7   62     81  2.59  50      0     1       0      695
## 2   5.1   59     66  1.70  39      0     0       0      403
## 3   7.4   57     83  2.16  55      0     0       0      710
## 4   6.5   73     41  2.01  48      0     0       0      349
## 5   7.8   65    115  4.30  45      0     0       1     2343
## 6   5.8   38     72  1.42  65      1     1       0      348
```

Part 1: Fit the linear model

```
fit.surgery <- lm(survival ~ ., data = surgery)

fit.surgery
```

```
##
## Call:
## lm(formula = survival ~ ., data = surgery)
##
## Coefficients:
## (Intercept)        blood         prog       enzyme        liver          age
##   -1148.823       62.390        8.973        9.888       50.413       -0.951
##      gender        a.mod      a.heavy
##      15.874        7.713      320.697
```

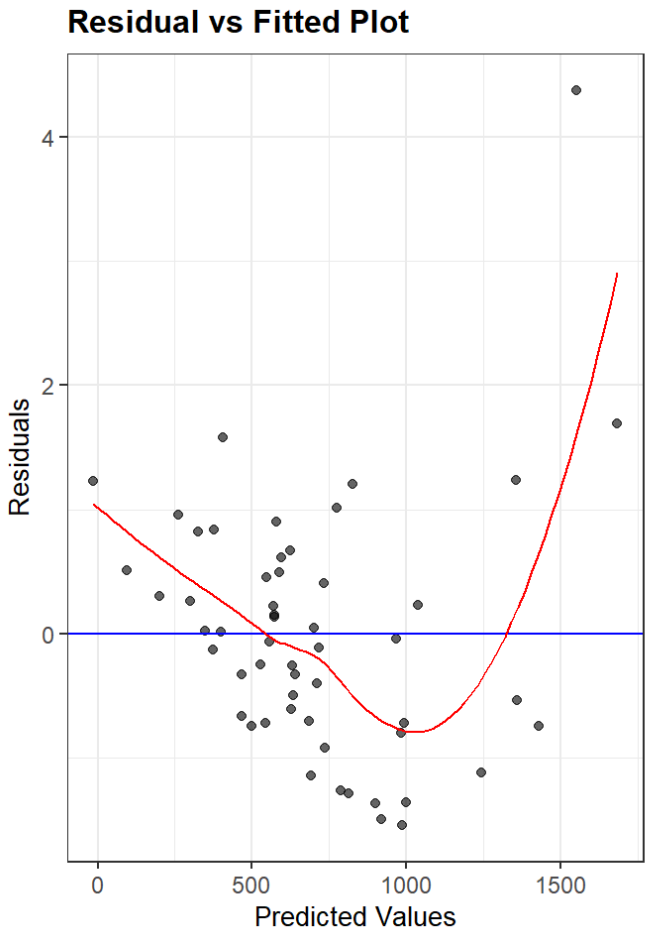
Part 2: Model Diagnostics 1

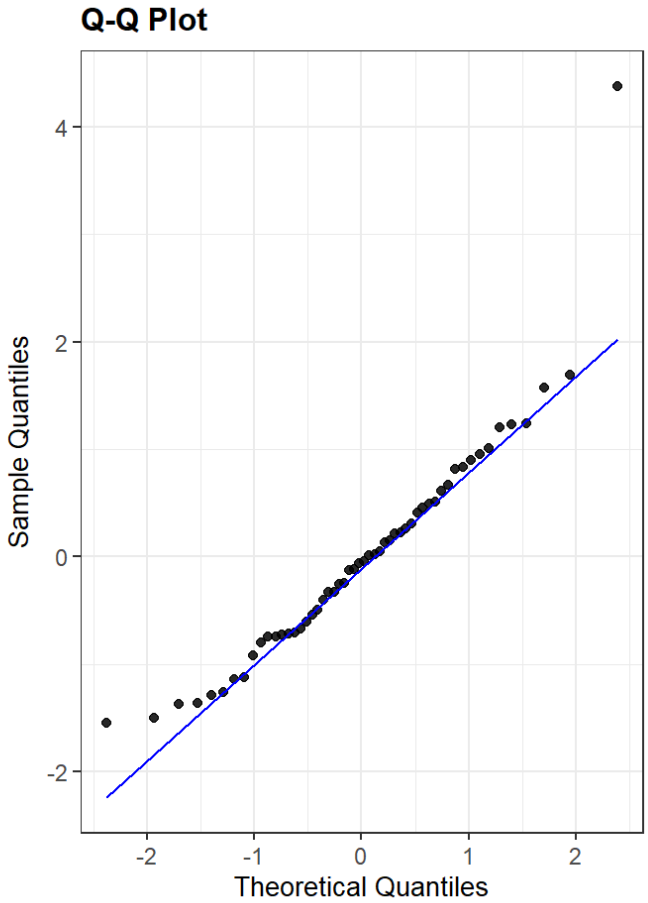
Check for Linearity, Normality and Constant Variance

- Linearity & Constant Variance -use Residual plot
- Normality -use Q-Q Plot

```
plot1 <- resid_panel(fit.surgery, plots = c('resid', 'qq'))

plot1
```





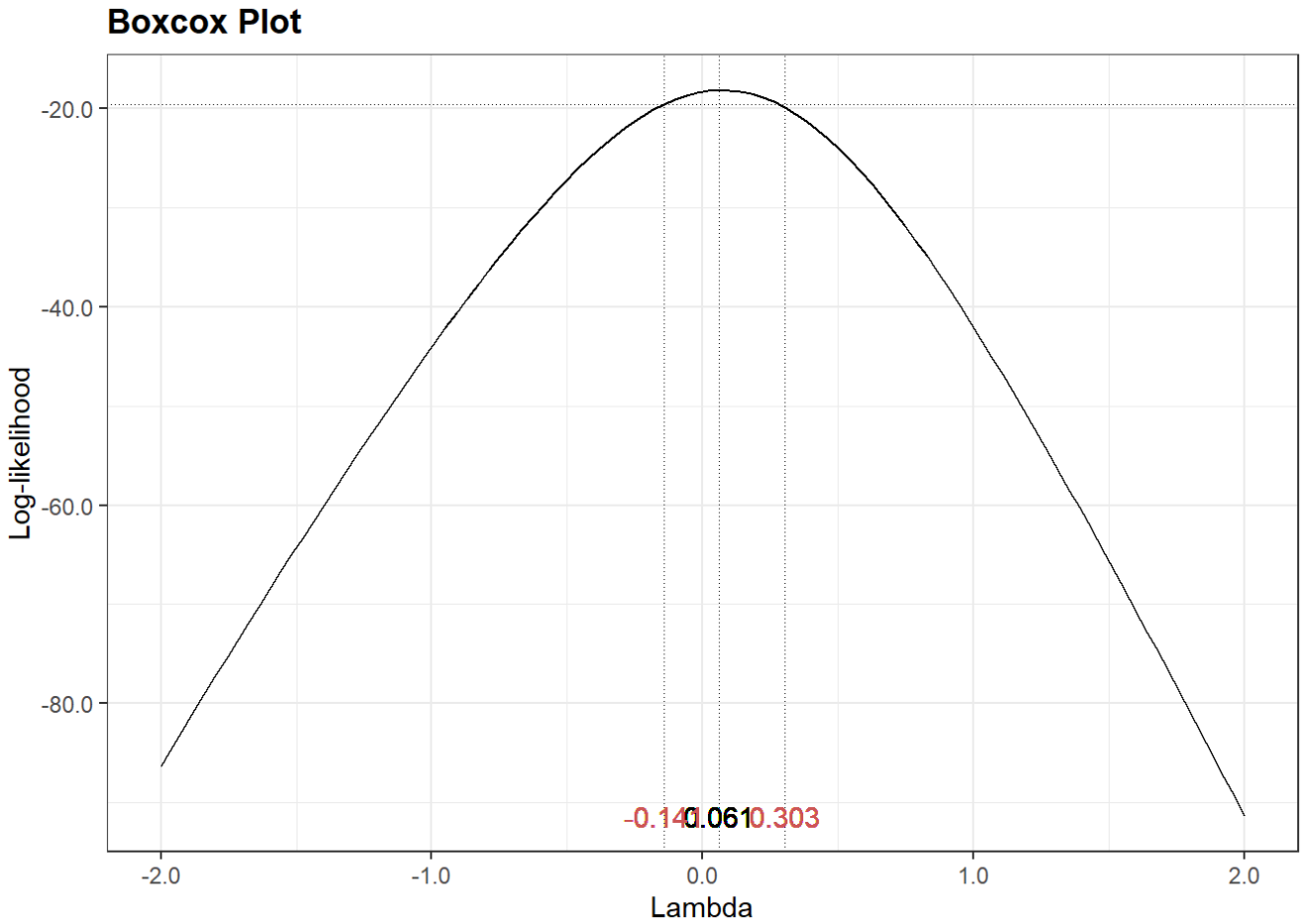
Results 2.1

- The **Linearity Assumption was not violated:** The residuals on the *Residual Plot* are not so far away from zero, by standardization, since majority of our values are between 2 and -2; then we are convinced there is a linear relationship between the predictors and the response variable.
- The **Normality assumption was not violated:** Most points follow the *Q-Q line*, with one clear outlier.
- The **Constant Variance assumption was violated:** From the *Residual Plot*, the residual spread is uneven across the predicted values (x-axis), and the large outliers makes it worse. **To fix this (unequal variance): transform the Y (response variable).** Note, this would also serve if normality is violated. We only transform X (i.e. predictor) when “linearity” is violated

Part 3A: Transform Y (response) using Box-Cox Transformation

```
library(lindia)

gg_boxcox(fit.surgery) +
  theme_bw() +
  theme(plot.title = element_text(face = "bold")) +
  scale_y_continuous(labels = label_number(accuracy = 0.1))  + # Y-axis to 1 dp
  scale_x_continuous(labels = label_number(accuracy = 0.1))   # X-axis to 1 dp
```



Results 3.1

- The lambda value is $ ~0.1$ with a 95% confidence interval of approximately -0.1 and 0.3. It makes more sense to take $\lambda = 0.0$ for better interpretation in practice.
- Since, $\lambda = 0$, that means our transformation becomes ** $ (Y) $** because:

$$T(Y) = \begin{cases} \frac{Y^{\lambda}-1}{\lambda} & \text{if } \lambda \neq 0 \\ \ln(Y) & \text{if } \lambda = 0 \end{cases}$$

Part 3B: Implement the Transformation of Y (response variable) in the dataset

```
surgery <- mutate(surgery, ln.survival = log(survival))

# check the new column
head(surgery)
```

```
##   blood prog enzyme liver age gender a.mod a.heavy survival ln.survival
## 1   6.7   62     81  2.59  50      0     1       0      695    6.543912
## 2   5.1   59     66  1.70  39      0     0       0      403    5.998937
## 3   7.4   57     83  2.16  55      0     0       0      710    6.565265
## 4   6.5   73     41  2.01  48      0     0       0      349    5.855072
## 5   7.8   65    115  4.30  45      0     0       1     2343    7.759187
## 6   5.8   38     72  1.42  65      1     1       0      348    5.852202
```

Part 4: Data Cleaning -remove the original response variable

```
surgery <- surgery %>% select(-survival)

# check to confirm
head(surgery)
```

```
##   blood prog enzyme liver age gender a.mod a.heavy ln.survival
## 1   6.7   62     81  2.59  50      0     1       0    6.543912
## 2   5.1   59     66  1.70  39      0     0       0    5.998937
## 3   7.4   57     83  2.16  55      0     0       0    6.565265
## 4   6.5   73     41  2.01  48      0     0       0    5.855072
## 5   7.8   65    115  4.30  45      0     0       1    7.759187
## 6   5.8   38     72  1.42  65      1     1       0    5.852202
```

Part 5: Re-fit the model using the transformed Y

```
# use the new response variable to fit the model

fit2.surgery <- lm(ln.survival ~., surgery)
fit2.surgery
```

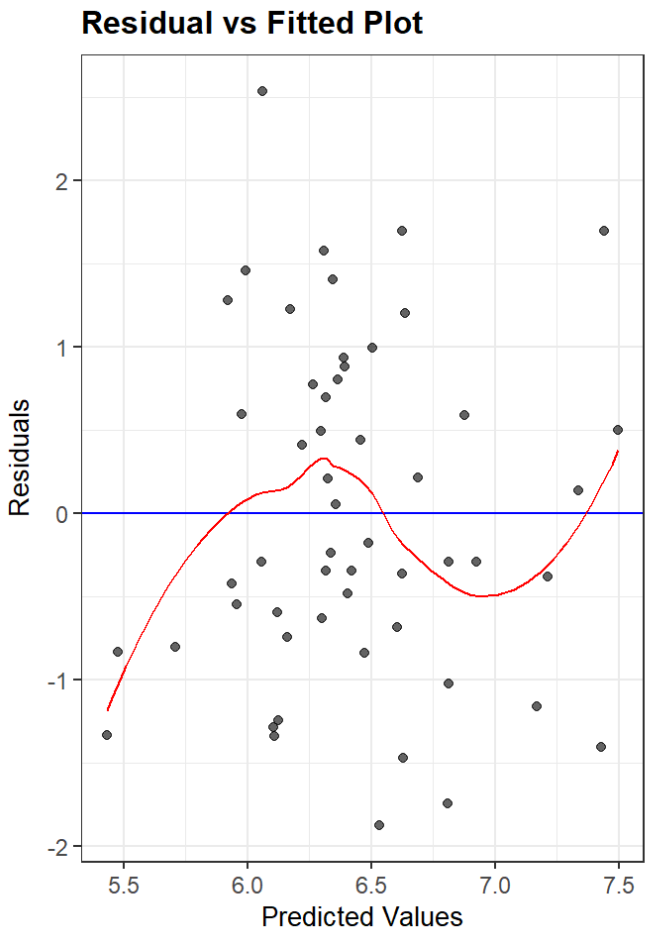
```
##
## Call:
## lm(formula = ln.survival ~ ., data = surgery)
##
## Coefficients:
## (Intercept)        blood         prog       enzyme        liver          age
##    4.050949     0.068551     0.013459     0.014948     0.007931    -0.003567
##      gender        a.mod      a.heavy
##    0.084151     0.057313     0.388190
```

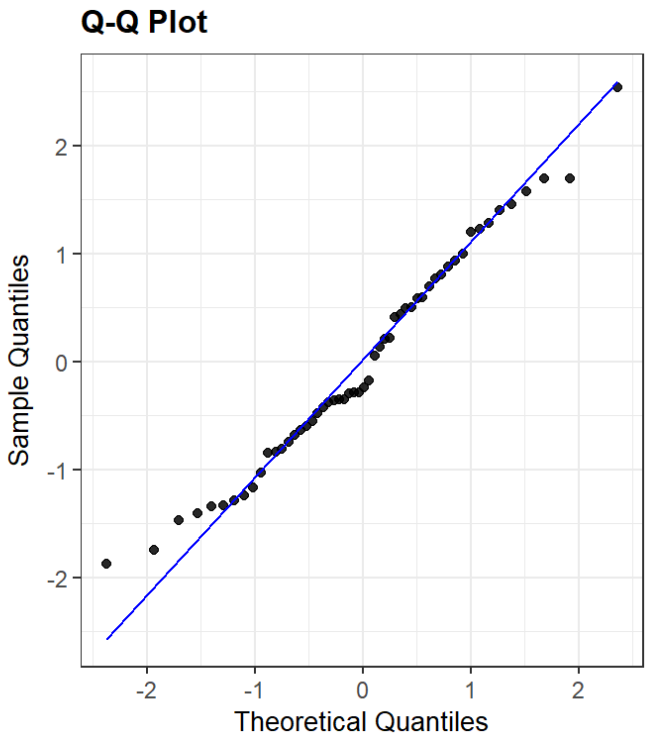
Part 6: Model Diagnostics 2

Re-check for Linearity, Normality and Constant Variance

```
plot2 <- resid_panel(fit2.surgery, plots = c('resid', 'qq'))

plot2
```





Results 6.1

- With this transformation, we see from the *Residual Plot* that the residual spread is even across the predicted values (x-axis), and there are no more outliers on the *Q-Q plots*.

Part 7: BEST MODEL SELECTION

Using “Best Subset Algorithm” Method

For context on this algorithm:

- The possible number of models based on our dataset is $2^{(p-1)} = 2^8 = 256$.

- Where $p = 9$, is the number of parameters (coefficients) in the full model, so, $p - 1 = 8$ (intercept excluded)
- As p increases, so does the multiple-coefficient-of-determination, R^2 thus, we should not pick the model with the largest R^2.
- We need the model with the least R^2.
- **GOAL:** Get a leveling off point where adding more variables provides little increase in R^2.

```
library(leaps)

select_model <- regsubsets(ln.survival ~ .,
                           data = surgery)


search.models = summary(select_model)
search.models
```

```
## Subset selection object
## Call: regsubsets.formula(ln.survival ~ ., data = surgery)
## 8 Variables  (and intercept)
##         Forced in Forced out
## blood       FALSE      FALSE
## prog        FALSE      FALSE
## enzyme      FALSE      FALSE
## liver       FALSE      FALSE
## age         FALSE      FALSE
## gender      FALSE      FALSE
## a.mod       FALSE      FALSE
## a.heavy     FALSE      FALSE
## 1 subsets of each size up to 8
## Selection Algorithm: exhaustive
##          blood prog enzyme liver age gender a.mod a.heavy
## 1  ( 1 ) " "   " "  "*"    " "   " " " " " "    " "   " "
## 2  ( 1 ) " "   "*"  "*"    " "   " " " " " "    " "   " "
## 3  ( 1 ) " "   "*"  "*"    " "   " " " " " "    " "   "*"
## 4  ( 1 ) "*"   "*"  "*"    " "   " " " " " "    " "   "*"
## 5  ( 1 ) "*"   "*"  "*"    " "   " " " " "*"    " "   "*"
## 6  ( 1 ) "*"   "*"  "*"    " "   "*" "*" "*"    " "   "*"
## 7  ( 1 ) "*"   "*"  "*"    " "   "*" "*" "*"    "*"   "*"
## 8  ( 1 ) "*"   "*"  "*"    "*"   "*" "*" "*"    "*"   "*"
```

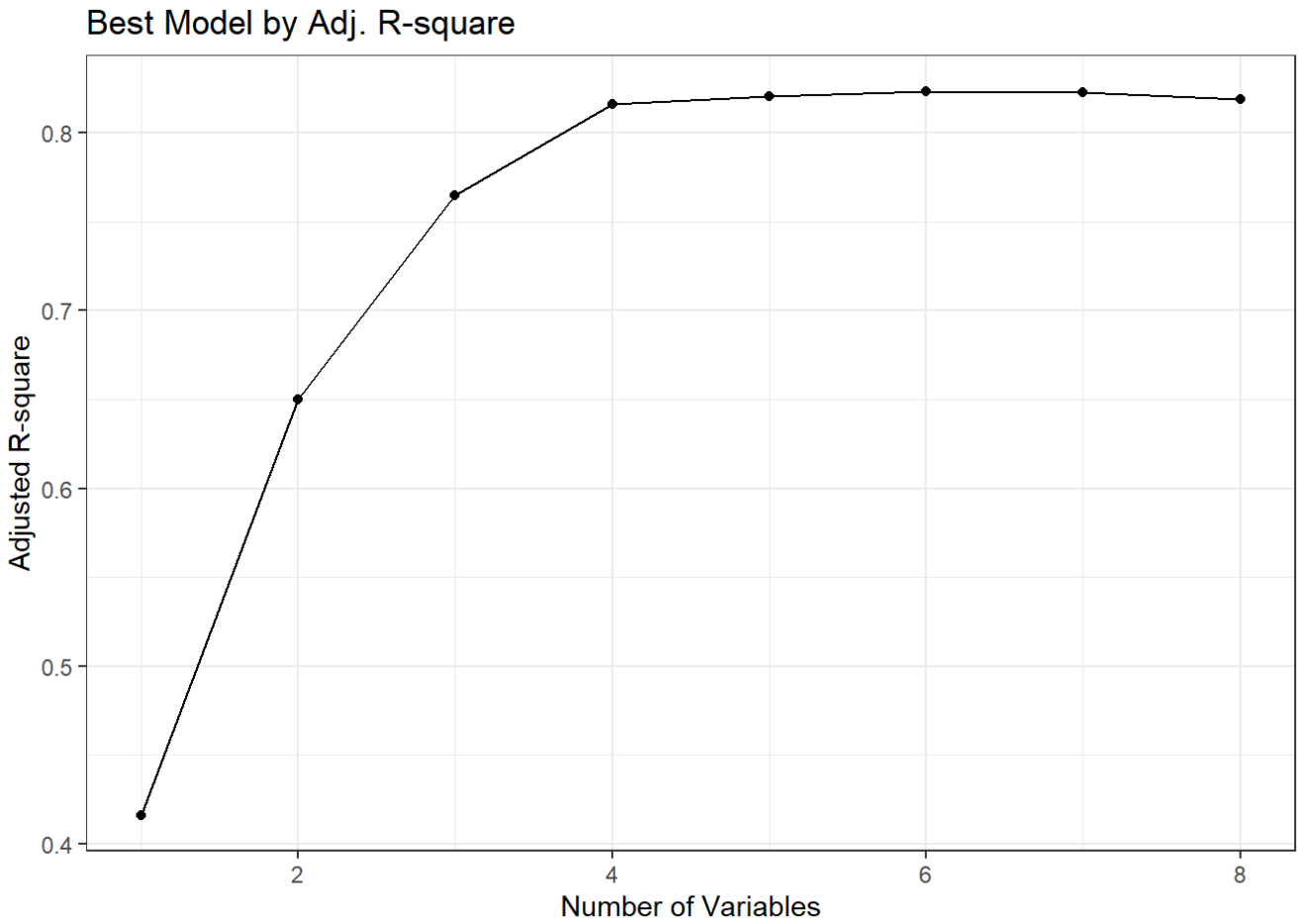
Results 7.1

- From the established *256 possible models*, the 8 models above are the best model with n predictors.
- For example, the *best model with 3 predictor variables has "prog", "enzyme" and "a.heavy"*

Which is the best model to use?

- Since R^2 will only increase as number of predictors increase, we would expect the model with all 8 parameters to have the highest R^2 and model with just "prog" to have the least.
- For this reason, we use the **adjusted R-squared** approach. This adjust more parameters to the regression model.
- Hence, we want the *model with the highest value of adjusted R-square*

```
tibble(predictors = seq(1, 8),
       y = search.models$adjr2) %>%
                                    ggplot(aes(x = predictors, y = y)) +
                                    geom_point() +
                                    geom_line() +
                                    labs(x = "Number of Variables", y = "Adjusted R-square",
                                         title = "Best Model by Adj. R-square") +
                                    # theme(plot.title = element_text(face = "bold")) +
                                    theme_bw()
```



Check the number of variables that has the highest *adjusted R-Square* in the graph

```
which.max(search.models$adjr2)
```

```
## [1] 6
```

Results 7.2

- **Best Model = model with 6 predictors**
- Thus, our best model, by *Best Subset Algorithm* will have the following predictors: $blood,\ prog,\ enzyme,\ age,\ gender,\ a.heavy$

Part 8: Refit the data into a new model

- Using only the relevant 6 predictors

```
best_model = lm(formula = ln.survival ~ blood + prog + enzyme + age + gender + a.heavy, data = surgery)

best_model
```

```
##
## Call:
## lm(formula = ln.survival ~ blood + prog + enzyme + age + gender +
##     a.heavy, data = surgery)
##
## Coefficients:
## (Intercept)        blood         prog       enzyme          age       gender
##    4.054389     0.071524     0.013760     0.015109    -0.003452     0.087227
##     a.heavy
##    0.351069
```

Observation and Next Steps

Although, our predictor variables are less than 30 for this dataset, I would like to test another method called the **Stepwise Regression Method**.

Part 9: Stepwise Regression Method

For context on this algorithm:

- When the number of variables is 30 or more, then *best subset algorithm* becomes computationally expensive.
- We will employ the **Backward Stepwise Search** (or simply backward elimination):
    - This elimination starts with the full model (all predictors) and then removes one variable at a time in the model.
    - It compares models using numerical criteria such as “AIC” (Akaike Information Criteria). The lower the better.

Using Backward Elimination

```
backward_elim <- step(object    = fit2.surgery,
                      direction = "backward")
```

```
## Start:  AIC=-160.78
## ln.survival ~ blood + prog + enzyme + liver + age + gender +
##     a.mod + a.heavy
##
##           Df Sum of Sq    RSS     AIC
## - liver    1   0.00126 1.9718 -162.74
## - a.mod    1   0.03159 2.0021 -161.92
## - age      1   0.07359 2.0441 -160.80
## <none>                 1.9705 -160.78
## - gender   1   0.08403 2.0545 -160.52
## - blood    1   0.31845 2.2890 -154.69
## - a.heavy  1   0.84489 2.8154 -143.51
## - prog     1   2.09285 4.0634 -123.70
## - enzyme   1   2.98863 4.9591 -112.94
##
## Step:  AIC=-162.74
## ln.survival ~ blood + prog + enzyme + age + gender + a.mod +
##     a.heavy
##
##           Df Sum of Sq    RSS      AIC
## - a.mod    1    0.0326 2.0043 -163.858
## <none>                 1.9718 -162.743
## - age      1    0.0876 2.0593 -162.396
## - gender   1    0.0969 2.0687 -162.152
## - blood    1    0.6269 2.5987 -149.835
## - a.heavy  1    0.8438 2.8156 -145.506
## - prog     1    2.6755 4.6473 -118.446
## - enzyme   1    5.0934 7.0652  -95.825
##
## Step:  AIC=-163.86
## ln.survival ~ blood + prog + enzyme + age + gender + a.heavy
##
##           Df Sum of Sq    RSS      AIC
## <none>                 2.0043 -163.858
## - age      1    0.0769 2.0812 -163.826
## - gender   1    0.0975 2.1018 -163.293
## - blood    1    0.6284 2.6327 -151.133
## - a.heavy  1    0.9011 2.9054 -145.810
```

```
## - prog     1    2.7644 4.7688 -119.052
## - enzyme   1    5.0752 7.0795  -97.716
```

Result 9.1

- The reduction stopped at 6 variables
- This implies that our BEST model is with the following predictors:
  $blood,\ prognostic\ index\ (prog.),\ enzyme,\ age,\ gender,\ heavy\ alcohol\ user\ (a.heavy)$
- **These predictors are the same the predictors we got when we used the Adjusted R-squared criteria.** Hence, the same *"best model."*

Part 10: Model Diagnostics 3

Check for multicollinearity of our *"Best Model"*

- Using the Variance Inflation Factor

$$VIF_i = \frac{1}{1 - R_i^2}$$

- VIF_i: The Variance Inflation Factor for a specific independent variable X_i.
- R_i^2: The coefficient of determination (R-squared value) obtained by regressing that specific independent variable X_i, against all other independent variables in the model.
- VIF is interpreted as:
  - $VIF = 1$: The variable is *completely uncorrelated* with other variables
  - $1 \leq VIF \leq 5$: The variable is *moderately correlated* with other variables
  - $VIF > 5\ or\ 10$: This is "severe multicollinearity". The coefficients for these variables may be poorly estimated, unstable, and difficult to interpret.

```
library(car)

vif(best_model)
```

```
##    blood     prog   enzyme      age   gender  a.heavy
## 1.108821 1.037099 1.076909 1.016471 1.047745 1.114506
```

Result 10.1

There are **no significant multicollinearity** in our model's predictor variables. Hence, **a good model.**